
Booting

From bootloader to scheduler

Introduction

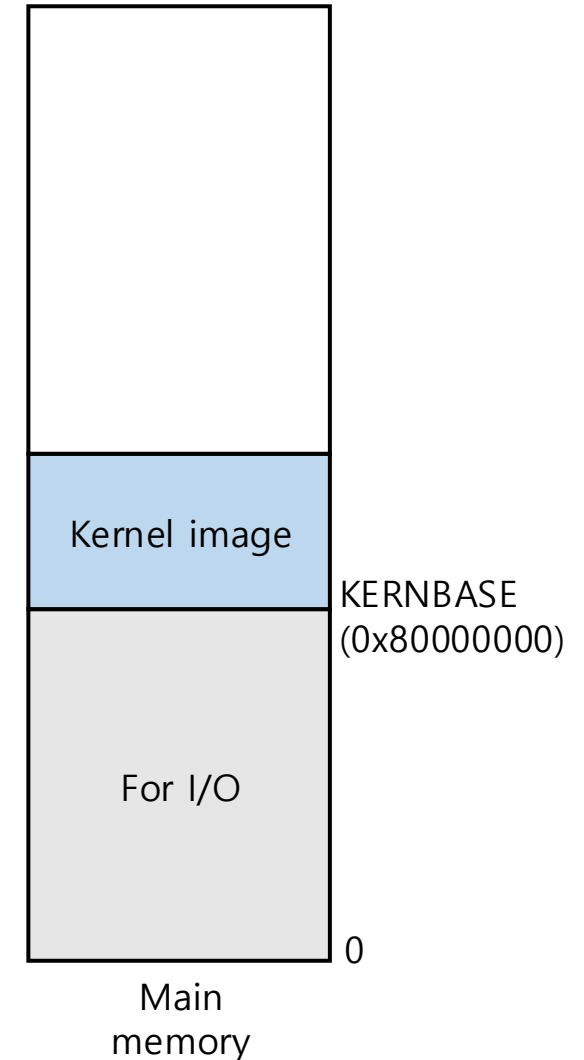
- We already know that user programs start from the `main()` function
- We also know that a process is created by `fork()` **system call** called from its parent process
- So, where is **xv6 kernel's entry point**? And when is the **first process created**?

Recall: Three RISC-V privilege modes

- Machine Mode
 - CPU starts in machine mode
 - Uses physical address space
- Supervisor Mode
 - Allowed to execute privileged instructions
 - The operating system kernel runs in supervisor mode
- User Mode
 - User processes run in user mode

Boot Loader

- When a RISC-V computer powers on, it begins execution from the **boot ROM**, which contains the **first-stage boot loader**
 - The bootloader loads the xv6 kernel into physical memory at address 0x80000000
 - This placement avoids the lower memory region(0x0 ~ 0x80000000), which typically reserved for I/O devices
- At this early stage, paging is disabled, so virtual addresses are identical to physical addresses
- In our case, **QEMU** acts as a boot loader



Kernel Initialization

```
$ vim kernel/kernel.ld
1  OUTPUT_ARCH( "riscv" )
2  ENTRY( _entry )
3
4  SECTIONS
5  {
6
7  ..
8  10      . = 0x80000000;
9  11
10 12      .text : {
11 13          kernel/entry.o(_entry)
12
13  ..
14 20      }
15 21
16 22      .rodata : {
17
18  ..
19 27      }
20 28
21 29      .data : {
22
23  ..
24 34      }
25 35
26 36      .bss : {
27
28  ..
29 41      }
30 42
31 43      PROVIDE(end = .);
32 44  }
```

- Once the build process completes, the **xv6 kernel binary** is generated
 - The structure of the kernel binary is organized according to the section definitions specified in **kernel/kernel.ld**
 - Including .text, .data, .bss, and .rodata
- Boot loader(QEMU) then loads this kernel binary into memory at physical address 0x80000000
- After loading, QEMU sets the program counter(PC) to 0x80000000, which points to the **_entry** symbol defined in **kernel/entry.S**
 - Execution begins from this entry point

Boot Entry

```
$ vim kernel/entry.S
```

```
5  .section .text
6  .global _entry
7  _entry:
8      # ① set up a stack for C
9      # stack0 is declared in start.c,
10     # with a 4096-byte stack per CPU.
11     # sp = stack0 + (hartid * 4096)
12     la sp, stack0
13     li a0, 1024*4
14     csrr a1, mhartid
15     addi a1, a1, 1
16     mul a0, a0, a1
17     add sp, sp, a0
18     # ② jump to start() in start.c
19     call start
20 spin:
21     j spin
```

- Place this code in .text and expose _entry globally
 - so that kernel/kernel.ld can use it as the entry point
- ① Set up the stack pointer to stack0
 - stack0[NCPU * 4096]: 4KB per hart, declared in start.c
- ② Call start() in C (still in Machine mode)
- Infinite loop as a fallback if start() ever returns

System Initialization

```
$ vim kernel/start.c

10 // entry.S needs one stack for CPU.
11 __attribute__((aligned(!^))) char stack0[4096 * NCPU];
12
13 // entry.S jumps here in machine mode on stack0.
14 void
15 start()
16 {
17     // ① set M Previous Privilege mode to Supervisor, for mret
18     unsigned long x = r_mstatus();
19     x &= ~MSTATUS_MPP_MASK;
20     x |= MSTATUS_MPP_S;
21     w_mstatus(x);
22
23     // ② set M Exception Program Counter to main, for mret
24     // requires gcc -mcmodel=medany
25     w_mepc((uint64)main);
26
27     // ③ disable paging for now.
28     w_satp(0);
```

- stack0: per-hart kernel stack (4KB, 16-byte aligned)
- ① MPP = Supervisor
→ mret will switch to S-mode
- ② MEPC = main
→ mret will jump to main()
- ③ Disable paging for supervisor mode (set up later)

System Initialization

```
$ vim kernel/start.c
```

```
...
30 // ④ delegate all interrupts and exceptions to supervisor mode.
31 w_medeleg(0xffff);
32 w_mideleg(0xffff);
33 w_sie(r_sie() | SIE_SEIE | SIE_STIE | SIE_SSIE);
34
35 // ⑤ configure Physical Memory Protection to give supervisor mode
36 // access to all of physical memory.
37 w_pmpaddr0(0x3fffffffffffffff);
38 w_pmpcfg0(0xf);
39
40 // ask for clock interrupts.
41 timerinit();
...
48 asm volatile("mret");
49 }
```

- mret is the key transition
 - Mode: Machine → Supervisor (from MPP)
 - PC: → main() (from MEPC)

- ④ Delegate traps to S-mode (medeleg, mideleg) and enable S-mode interrupts (sie)
- ⑤ PMP: allow Supervisor to access all physical memory
- timerinit(): schedule the first timer interrupt

Overview: Main function of kernel

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
14         ...
17         printf("xv6 kernel is booting\n");
19         kinit();
20         kvminit();
21         kvminithart();
22         procinit();
23         trapinit();
24         trapinithart();
25         plicinit();
26         plicinithart();
27         binit();
28         iinit();
29         fileinit();
30         virtio_disk_init();
31         userinit();
32         __sync_synchronize();
33         started = 1;
34     } else {
35         while(started == 0)
36             ;
37         __sync_synchronize();
38         printf("hart %d starting\n", cpuid());
39         kvminithart();
40         trapinithart();
41         plicinithart();
42     }
44     scheduler();
45 }
```

- The kernel begins execution at the main function
- CPU 0 performs system-wide initialization of all kernel subsystems
 - Other CPUs wait until initialization is complete
- All CPUs then enter the scheduler to execute user processes

Boot Synchronization: CPU 0 vs Others

```
$ vim kernel/main.c

10 void
11 main()
12 {
13     if(cpuid() == 0){
14         ...
17         printf("xv6 kernel is booting\n");
18         kinit();
19         kvminit();
20         kvminithart();
21         procinit();
22         trapinit();
23         trapinithart();
24         plicinit();
25         plicinithart();
26         binit();
27         iinit();
28         fileinit();
29         virtio_disk_init();
30         userinit();
31         __sync_synchronize();
32         started = 1;
33     } else {
34         while(started == 0)
35             ;
36         __sync_synchronize();
37         printf("hart %d starting\n", cpuid());
38         kvminithart();
39         trapinithart();
40         plicinithart();
41     }
42     scheduler();
43 }
```

- hart = Hardware Thread, RISC-V's term for a CPU core's execution context
 - QEMU spawns N harts with `-smp N` option
 - Every hart runs the same `main()` independently
- CPU 0 performs global initialization, others wait
 - `started`: a flag set by CPU 0 to signal "init done"
 - `__sync_synchronize()`: memory barrier
- Naming convention
 - `xxxinit()` → called once (global state)
 - `xxxinithart()` → called per-hart (touches hart-local CSRs)

Verifying Multi-hart Boot

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
17         printf("xv6 kernel is booting\n");
32         __sync_synchronize();
33         started = 1;
34     } else {
35         while(started == 0)
36             ;
37         __sync_synchronize();
38         printf("hart %d starting\n", cpuid());
42     }
```

- We'll see how this process is created next

```
~/o/xv6-riscv-sandbox-for-TA sandbox/base make qemu CPUS=3 14:26:35
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

```
xv6 kernel is booting
```

```
hart 1 starting
```

```
hart 2 starting
```

```
init: starting sh
```

```
$
```

Initialize physical page

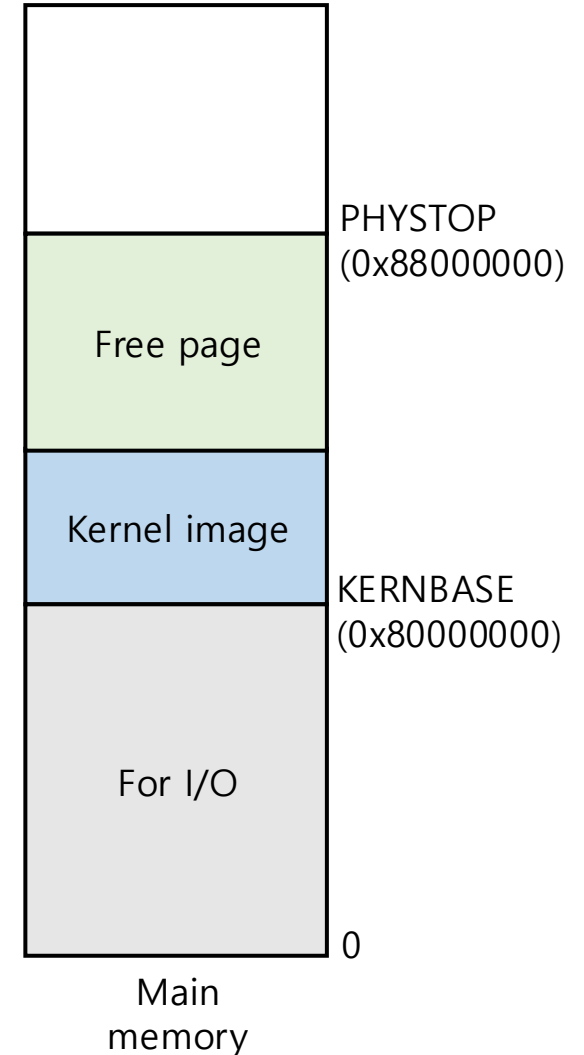
```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
...
19     kinit();
...
45 }
```

```
$ vim kernel/kalloc.c
```

```
26 void
27 kinit()
28 {
29     initlock(&kmem.lock, "kmem");
30     freerange(end, (void*)PHYSTOP);
31 }
32
33 void
34 freerange(void *pa_start, void *pa_end)
35 {
36     char *p;
37     p = (char*)PGROUNDUP((uint64)pa_start);
38     for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE)
39         kfree(p);
40 }
```

The range from end to PHYSTOP is added to the free list through the **freerange()** function.



Initialize Kernel Page Table

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
14         ...
19         kinit();
20         kvminit();
21         kvmminithart();
22         procinit();
23         trapinit();
24         trapinithart();
25         plicinit();
26         plicinithart();
27         binit();
28         iinit();
29         fileinit();
30         virtio_disk_init();
31         userinit();
32         __sync_synchronize();
33         started = 1;
34     } else {
35         ...
42     }
43
44     scheduler();
45 }
```

- Sets up kernel page table by calling `kvminit()` and `kvmminithart()`
- These functions were covered in VM lecture

Recall: Initialize Kernel Page Table

```
$ vim kernel/vm.c
```

```
52 // Initialize the one kernel_pagetable
53 void
54 kvminit(void)
55 {
56     kernel_pagetable = kvmmake();
57 }
```

```
$ vim kernel/vm.c
```

```
61 void
62 kvminithart()
63 {
64     // wait for any previous writes to ...
65     sfence_vma();
66
67     w_satp(MAKE_SATP(kernel_pagetable));
68
69     // flush stale entries from the TLB.
70     sfence_vma();
71 }
```

- `kvminit()`: build the kernel page table
 - Direct mapping: kernel code/data, devices, trampoline
 - Internally calls `kvmmake()`
- `kvminithart()`: activate it on this hart
 - Write to `satp` register
 - Flush TLB with `sfence_vma()`
- Detailed mapping logic was covered in the VM lecture

Initialize process table

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
...
22     procinit();
...
```

```
$ vim kernel/proc.c
```

```
47 void
48 procinit()
49 {
50     struct proc *p;
51
52     initlock(&pid_lock, "nextpid");
53     initlock(&wait_lock, "wait_lock");
54     for(p = proc; p < &proc[NPROC]; p++) {
55         initlock(&p->lock, "proc");
56         p->state = UNUSED;
57         p->kstack = KSTACK((int) (p - proc));
58     }
59 }
```

- **procinit()** initializes the process table by iterating each entry, setting up
 - Each process's lock
 - Initializing its state to UNUSED
 - Assigning the starting address of its kernel stack

Initialize trap

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
...
23     trapinit();
24     trapinithart();
```

```
$ vim kernel/trap.c
```

```
19 void
20 trapinit()
21 {
22     initlock(&tickslock, "time");
23 }
24
25 // set up to take exceptions and traps while ..
26 void
27 trapinithart(void)
28 {
29     w_stvec((uint64)kernelvec);
30 }
```

- **trapinit()** initializes a spinlock to protect access to the system timer tick counter(ticks)
- **trapinithart()** sets the trap vector address for the current hardware thread to the kernel trap handler
 - The **stvec** register holds the trap vector address, and the CPU jumps to this address when a trap occurs

Initialize interrupt

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuuid() == 0){
...
25     plicinit();
26     plicinithart();
```

- PLIC (Platform-Level Interrupt Controller) manages external device interrupts
- `plicinit()`: set per-device IRQ(Interrupt request) priority (system-wide)
- `plicinithart()`: enable IRQs this hart will receive (per-hart)
- After this, the system can receive UART and disk interrupts

```
$ vim kernel/plic.c
```

```
11 void
12 plicinit()
13 {
14     // set desired IRQ priorities non-zero (otherwise disabled).
15     *(uint32*)(PLIC + UART0_IRQ*4) = 1;
16     *(uint32*)(PLIC + VIRTIO0_IRQ*4) = 1;
17 }
18
19 void
20 plicinithart(void)
21 {
22     int hart = cpuuid();
26     *(uint32*)PLIC_SENABLE(hart) = (1 << UART_IRQ) | (1 << VIRTIO0_IRQ);
29     *(uint32*)PLIC
30 }
```

Initialize file system

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
14         ..
15         kinit();
16         kvminit();
17         kvmminithart();
18         procinit();
19         trapinit();
20         trapinithart();
21         plicinit();
22         plicinithart();
23         binit();
24         iinit();
25         fileinit();
26         virtio_disk_init();
27         userinit();
28         __sync_synchronize();
29         started = 1;
30     } else {
31         ..
32     }
33     scheduler();
34 }
```

- **binit()** prepares buffers used to temporarily store disk block data in memory
- **iinit()** initializes data structures for managing inodes(file metadata) in memory
 - It represents their state on disk
- **fileinit()** sets up the global file table to track all open files in the system
- **virtio_disk_init()** initializes the VirtIO disk device used in the QEMU virtualized environment

Allocate the First Process

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
...
31     userinit();
```

```
$ vim kernel/proc.c
```

```
219 void
220 userinit()
221 {
222     struct proc *p;
223
224     p = allocproc();
225     initproc = p;
226
227     p->cwd = namei("/");
228
229     p->state = RUNNABLE;
230
231     release(&p->lock);
232 }
```

- userinit() is called once by CPU 0 during boot
- It does **not** run /init itself. It just **prepares** the process.
 - ① allocproc() reserves a slot in the process table
 - ② p->context.ra = forkret
 - ③ p->state = RUNNABLE : the process is now eligible for scheduling
- The scheduler will pick it up later (next slide)

```
$ vim kernel/proc.c
```

```
109 static struct proc*
110 allocproc(void)
111 {
...
143     // Set up new context to start executing at forkret,
144     // which returns to user space.
145     memset(&p->context, 0, sizeof(p->context));
146     p->context.ra = (uint64)forkret;
147     p->context.sp = p->kstack + PGSIZE;
148
149     return p;
150 }
```

Where /init Actually Starts

```
$ vim kernel/proc.c
```

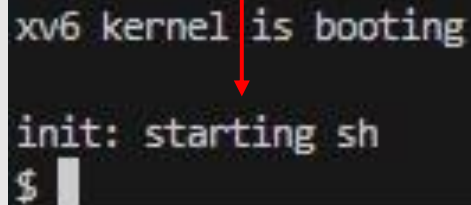
```
505 void
506 forkret(void)
507 {
508     extern char userret[];
509     static int first = 1;
510     struct proc *p = myproc();
511     ...
512     release(&p->lock);
513     ...
514     if(first) {
515         ...
516         fsinit(ROOTDEV);
517         ...
518         first = 0;
519         __sync_synchronize();
520         ...
521         p->trapframe->a0 = kexec("/init", (char *[]){ "/init, 0 });
522     }
523     ...
524     prepare_return();
525     uint64 satp = MAKE_SATP(p->pagetable);
526     uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
527     ((void (*)(uint64))trampoline_userret)(satp);
528 }
```

- Scheduler picks the RUNNABLE initproc, calls `swtch()` into it
- Because `ra == forkret`, execution lands in `forkret()`
- Initialize file system (`fsinit()`), then `kexec("/init")` replaces the process image with `/init`
- `static int first` ensures this one-time setup runs only when `initproc` is scheduled for the first time

initprocess

```
$ vim user/init.c
```

```
14  int
15  main(void)
16  {
17      int pid, wpid;
...
26      for(;;){
27          printf("init: starting sh\n");
28          pid = fork();
...
33          if(pid == 0){
34              exec("sh", argv);
35              printf("init: exec sh failed\n");
36              exit(1);
37          }
38
39          for(;;){
40              // this call to wait() returns if the shell exits,
41              // or if a parentless process exits.
42              wpid = wait((int *) 0);
43              if(wpid == pid){
44                  // the shell exited; restart it.
45                  break;
46              } else if(wpid < 0){
...
52      }
53  }
54 }
```



```
xv6 kernel is booting
init: starting sh
$ █
```

- Initprocess's main repeatedly runs the **shell(sh)** in an infinite loop
 - It creates a child process using **fork()**, and that child executes "**sh**" via **exec()**
- Meanwhile, the **parent process(initproc)** monitors and the **child(sh)** to see if it exits
 - If the shell exits, the loop breaks and restarts the shell again

Launch scheduler

```
$ vim kernel/main.c
```

```
10 void
11 main()
12 {
13     if(cpuid() == 0){
14         ..
19     kinit();
20     kvminit();
21     kvminithart();
22     procinit();
23     trapinit();
24     trapinithart();
25     plicinit();
26     plicinithart();
27     binit();
28     iinit();
29     fileinit();
30     virtio_disk_init();
31     userinit();
32     __sync_synchronize();
33     started = 1;
34     } else {
35         ..
42     }
43
44     scheduler();
45 }
```

- Finally, main calls **scheduler()** to launch the infinite process scheduling loop
 - If finds runnable processes and dispatches them to the CPU
 - Once entered, the kernel never returns from this point

Thank you

